

# Préparation à la soutenance — questions du jury et réponses

Ce document contient : le déroulé de présentation conseillé, les questions les plus probables avec des réponses prêtes, et les pièges à éviter.

## SOMMAIRE

- 1. [Le déroulé de présentation conseillé](#)
- 2. [Questions sur l'architecture générale](#)
- 3. [Questions sur le backend](#)
- 4. [Questions sur la sécurité](#)
- 5. [Questions sur les frontends](#)
- 6. [Questions sur l'IA](#)
- 7. [Questions sur le métier \(paie, TVA, congés\)](#)
- 8. [Questions sur la méthode et la gestion de projet](#)
- 9. [Questions pièges et comment y répondre](#)
- 10. [Les limites à assumer](#)
- 11. [Fiche mémo : les chiffres à connaître par cœur](#)
- 12. [Checklist avant la soutenance](#)

## 1. Le déroulé de présentation conseillé

### Structure en 20 minutes

| Temps | Contenu                                                                      | Objectif                                    |
|-------|------------------------------------------------------------------------------|---------------------------------------------|
| 2 min | Le contexte : Antigone, agence de communication, 3 métiers dispersés         | Poser le <b>problème</b> avant la solution  |
| 3 min | L'architecture : 1 backend, 3 frontends, 1 agent, 1 assistant IA + le schéma | Donner la <b>carte</b>                      |
| 3 min | Le modèle de sécurité : Employé → Compte → Rôle → Permission                 | C'est le <b>fil rouge</b> de tout le projet |
| 5 min | <b>Démonstration live</b> (voir scénario ci-dessous)                         | Montrer que <b>ça marche</b>                |
| 4 min | Les 3 points techniques forts (au choix, voir §1.3)                          | Montrer la <b>profondeur</b>                |
| 2 min | Limites, perspectives, bilan                                                 | Montrer la <b>maturité</b>                  |
| 1 min | Conclusion                                                                   |                                             |

### 1.2 Le scénario de démonstration (5 minutes)

Un fil narratif est infiniment plus convaincant qu'un tour des menus.

1. Connexion en tant qu'ADMIN sur l'application Projets

↓

2. Ouvrir un client (Nike) → montrer la fiche marque complète  
« Ces champs identité/activité/positionnement ne sont pas décoratifs : ils alimentent le RAG de l'IA. »

↓

3. Aller sur Media Plan → ouvrir l'ASSISTANT IA

« Génère le media plan de Nike pour octobre »  
→ MONTRER LES ÉVÉNEMENTS D'OUTILS s'afficher en direct  
(« Recherche de la marque... », « Historique... », « Génération... »)  
→ les cartes de publications apparaissent  
« Rien n'est enregistré : je dois valider moi-même. »  
↓  
4. Cliquer sur AppSwitchButton → passer à l'application RH  
« SANS ME RECONNECTER. Le jeton transite par window.name, la seule propriété qui survit à une navigation inter-origines. »  
↓  
5. Sur RH : Employés → montrer la fiche complète, le champ modeSalaire = NET  
↓  
6. Passer à l'application FINANCE  
→ Salaires → générer les bulletins du mois  
→ ouvrir le détail d'un bulletin : montrer la CHAÎNE DE CALCUL COMPLÈTE  
« L'employé avait un salaire NET négocié. Le système a retrouvé le brut par recherche dichotomique, parce que l'IRPP progressif n'est pas inversible analytiquement. »  
↓  
7. Factures → une facture en retard → bouton « Relancer »  
→ l'assistant s'ouvre avec la question pré-remplie  
→ il produit un BROUILLON avec le ton imposé par les jours de retard  
« Le ton n'est pas choisi par l'IA : ≤7 jours SOFT, ≤30 FIRM, au-delà FORMAL. Et rien n'est envoyé tant que je n'ai pas cliqué. »  
↓  
8. (si le temps le permet) Ouvrir Swagger UI  
« Voilà les 40 contrôleurs et l'ensemble de l'API. »

### 1.3 Les 3 points techniques forts — choisissez selon le jury

| Point fort                                                  | Pour quel jury                                     | Où c’est documenté     |
|-------------------------------------------------------------|----------------------------------------------------|------------------------|
| A. L’inversion NET → BRUT par dichotomie                    | Jury orienté algorithmique / mathématiques         | 01-BACKEND.md §12.3    |
| B. L’isolation par instance d’outil IA (pas de ThreadLocal) | Jury orienté architecture logicielle / concurrence | 05-IA...md §6          |
| C. Le RAG hybride avec fusion RRF                           | Jury orienté IA / recherche d’information          | 05-IA...md §7          |
| D. Le SSO par window.name                                   | Jury orienté web / sécurité                        | 00-VUE-DENSEMBLE.md §5 |
| E. Le modèle rôles/permissions dynamique                    | Jury orienté systèmes d’information                | 00-VUE-DENSEMBLE.md §4 |

Conseil : préparez-en **trois** solidement, mentionnez les autres en une phrase.

## 2. Questions sur l’architecture générale

? « Pourquoi trois frontends et pas une seule application ? »

« Trois raisons. D’abord la séparation métier : un comptable ne doit jamais voir la page Media Plan, un social media manager ne doit jamais voir les salaires. Séparer physiquement les applications rend l’erreur impossible, alors qu’un seul gros front reposerait uniquement sur des conditions dans le code. Ensuite le poids : l’application Finance a 7 dépendances contre 13 pour Projets — elle ne charge ni Three.js, ni Ant Design, ni moment. Enfin le déploiement : les trois sont construites séparément, une régression sur l’une ne bloque pas les autres. Et elles partagent le même backend, le même JWT et le même compte : on passe de l’une à l’autre sans se reconnecter. »

? « Pourquoi un monorepo ? »

« Pour partager du code sans publier de package npm. Le widget de chat IA est un workspace @antigone/ai-chat-widget lié localement : je le modifie une fois, les trois applications en bénéficient immédiatement. Les dépendances communes sont aussi dédoublées dans un seul node\_modules. »

? « Pourquoi Spring Boot et pas Node.js / Django ? »

« Trois raisons. Le métier est fortement transactionnel — paie, facturation, validation de congés : JPA et @Transactional donnent des garanties fortes. Le typage statique de Java attrape à la compilation des erreurs qu’un langage dynamique ne révélerait qu’en production, ce qui est important sur un calcul de paie. Et l’écosystème Spring couvrirait tous mes besoins de façon intégrée : Security, Data JPA, Mail, Scheduling, et LangChain4j pour l’IA. »

## ? « Pourquoi PostgreSQL ? »

« Trois besoins concrets. La recherche plein texte native avec le dictionnaire français ( `to_tsvector('french', ...)` ), utilisée par la branche lexicale du RAG. L'extension `pgvector` pour la recherche vectorielle. Et le support natif du JSON pour stocker les lignes de facture et les éléments de salaire sans multiplier les tables. MySQL n'offre pas la même qualité de recherche textuelle multilingue. »

## ? « Comment les trois applications partagent-elles la session ? »

« Par un relais via `window.name`. Le `localStorage` est isolé par origine, donc inutilisable entre les trois. Avant de rediriger, l'application encode le jeton en Base64 dans `window.name` — l'une des rares propriétés qui survivent à une navigation inter-origines dans le même onglet. L'application cible le lit, l'enregistre dans son propre `localStorage`, puis **efface** `window.name` pour ne pas laisser le jeton exposé. »

## 3. Questions sur le backend

### ? « Décrivez votre architecture en couches. »

« Quatre couches strictes. Le contrôleur reçoit un DTO, valide, délègue — il ne contient aucune logique métier. Le service porte toute la logique, gère les transactions et convertit entité ↔ DTO. Le repository accède aux données via Spring Data. L'entité mappe la table. Plus, en amont de tout, un filtre JWT et une chaîne de sécurité Spring. »

### ? « Pourquoi des DTOs plutôt que d'exposer les entités ? »

« Trois raisons concrètes dans mon projet. D'abord ne jamais exposer le `passwordHash` : `CompteDTO` ne le contient pas. Ensuite éviter les boucles de sérialisation — `Employe` référence `Compte` qui référence `Employe`. Enfin éviter le `LazyInitializationException` : mes relations sont en `FetchType.LAZY`, et la conversion en DTO se fait à l'intérieur de la transaction. »

### ? « Pourquoi de l'héritage JPA pour les demandes ? »

« Une demande de congé, une autorisation de sortie et une demande de télétravail partagent exactement le même cycle de validation — créer, en attente, approuver ou refuser, historiser — mais ont des champs différents. J'ai utilisé la stratégie JOINED : une table mère `demandes` et une table fille par sous-type. Ça factorise le cycle de vie sans polluer les tables de colonnes nulles, ce qu'aurait fait la stratégie SINGLE\_TABLE. »

### ? « Comment gérez-vous les migrations de schéma ? »

« `ddl-auto: update` en développement. Je sais que ce n'est pas une solution de production : Hibernate ajoute des colonnes mais ne supprime jamais rien et ne modifie pas une contrainte existante. J'ai donc une classe `SchemaCleanup` pour les opérations qu'il ne sait pas faire. La bonne réponse en production serait **Flyway** ou **Liquibase**, avec des scripts de migration versionnés — c'est l'évolution que j'identifie en priorité. »

*(Assumer cette limite est meilleur que de la défendre.)*

### ? « À quoi servent les convertisseurs JSON ? »

« Trois `AttributeConverter` stockent des listes en colonne texte JSON : les lignes de facture, les éléments de salaire, les tranches du barème IRPP. Ces listes n'ont aucune valeur en dehors de leur parent — on ne requête jamais "toutes les lignes de facture de mars". Créer une table, une relation et une cascade de suppression aurait été de la complexité gratuite. »

### ? « Pourquoi persister toutes les étapes du calcul de paie ? »

« Parce qu'un bulletin doit rester justifiable et reproductible même si les taux changent l'année suivante. Je stocke le brut, le brut effectif, la CNSS, l'imposable, l'abattement, le revenu net imposable, la solidarité, l'IRPP, le net et le net à payer. Effet de bord très utile : l'IA peut expliquer un bulletin ligne par ligne **sans jamais recalculer**, elle ne fait que reformuler des valeurs persistées. »

#### ? « Vous avez des tâches planifiées, lesquelles ? »

« Sept. Toutes les 2 h, les alertes sur les tâches en dépassement. Tous les jours à 8 h, les rappels d'échéance et l'alerte sur les contrats arrivant à terme. Toutes les heures, l'archivage des tâches terminées. Toutes les 5 minutes, la fermeture automatique des pointages restés ouverts. Le 1er janvier, la création de l'arborescence Drive de l'année. Et à 3 h du matin, la réindexation du RAG. »

## 4. Questions sur la sécurité

#### ? « Expliquez votre système d'authentification. »

« JWT stateless signé en HS256, valable 12 heures. À la connexion, je vérifie le mot de passe avec BCrypt, puis je génère un jeton qui contient l'identifiant du compte, celui de l'employé, ses rôles **et ses permissions**. À chaque requête, un `OncePerRequestFilter` parse le jeton et pose l'`Authentication` dans le `SecurityContextHolder`. Aucune session serveur : `SessionCreationPolicy.STATELESS`. »

#### ? « Pourquoi mettre les permissions dans le jeton ? »

« Pour zéro requête SQL au moment d'autoriser. L'inconvénient, que j'assume, est qu'un changement de droits ne prend effet qu'à la reconnexion — au plus 12 heures. C'est le compromis classique du stateless. Si l'application devait exiger une révocation immédiate, il faudrait soit raccourcir la durée du jeton avec un refresh token, soit maintenir une liste de révocation côté serveur — ce qui réintroduirait de l'état. »

#### ? « Comment gérez-vous les permissions côté frontend ? »

« Le frontend cache, le backend interdit. Le frontend masque les menus et bloque les routes avec `PermissionRoute`, mais c'est du confort utilisateur, pas de la sécurité — n'importe qui peut modifier le JavaScript dans son navigateur. La sécurité réelle est côté serveur : `SecurityConfig` filtre par URL, et `@PreAuthorize` filtre au niveau des méthodes. »

#### ? « Quel est le rôle de `@EnableMethodSecurity` ? »

« Sans cette annotation, **toutes les annotations `@PreAuthorize` du projet sont ignorées silencieusement** — elles ne lèvent aucune erreur, elles ne sont simplement jamais évaluées. C'est un piège classique de Spring Security. Elle est indispensable au cloisonnement de l'assistant IA. »

#### ? « Comment protégez-vous les mots de passe ? »

« BCrypt via `BCryptPasswordEncoder`. C'est une fonction de hachage lente avec sel intégré — le sel est généré aléatoirement et stocké dans le hash lui-même, donc deux utilisateurs avec le même mot de passe ont des hash différents. La lenteur est délibérée : elle rend les attaques par force brute impraticables. »

#### ? « Et le portail client, c'est risqué non ? »

« C'est effectivement le point le plus exposé, puisque des personnes extérieures à l'entreprise s'y connectent. J'ai donc empilé quatre contrôles indépendants. Un : authentification par jeton. Deux : le type de principal doit être CLIENT. Trois : la page demandée doit figurer dans la liste `clientPages` du client en base. Quatre — et c'est le plus important — l'accès Google

Drive du portail passe par un **compte de service distinct, en lecture seule** ( `DRIVE_READONLY` ). Même en cas de compromission du portail, un client ne peut rien modifier ni supprimer sur le Drive de l'agence. »

## ? « Avez-vous pensé aux vulnérabilités connues ? »

« Oui, j'ai fait un audit et j'ai surchargé les versions transitives concernées dans le `pom.xml`, en documentant chaque CVE en commentaire : Tomcat 10.1.53, Spring Security 6.5.9, le driver PostgreSQL 42.7.11, json-smart 2.5.2 et Guava 32.1.3. Ces versions corrigent une douzaine de CVE identifiées. »

## ? « Y a-t-il des secrets dans votre dépôt ? »

Question probable si le jury regarde le code. Répondez avant qu'on vous le demande :

« J'ai identifié un problème : une clé API OpenAI s'est retrouvée écrite en dur dans `application.yml` au lieu d'utiliser `${OPENAI_API_KEY:}` comme partout ailleurs dans le fichier. Comme le fichier est versionné, la clé est présente dans l'historique Git et doit être considérée comme compromise. La remédiation est en trois temps : révoquer la clé sur la console OpenAI, en générer une nouvelle, et remplacer la ligne par la variable d'environnement — qui est d'ailleurs déjà déclarée dans `render.yaml` avec `sync: false`. C'est une bonne illustration du fait que la gestion des secrets doit être systématique, pas ponctuelle. »

Signaler soi-même une faiblesse et connaître sa remédiation vaut toujours mieux que de se la faire signaler.

## 5. Questions sur les frontends

### ? « Pourquoi React et pas Angular ou Vue ? »

« React 19 pour l'écosystème et pour les hooks, qui permettent de factoriser la logique — j'ai `useTheme`, `useSidebar`, `useConfirm`, `useTachesObligatoires`. Angular aurait été plus structurant mais aussi beaucoup plus verbeux pour trois applications de tailles très différentes. Et React s'intègre naturellement avec le widget partagé, qui est lui-même un composant React. »

### ? « Comment gérez-vous l'état global ? »

« Par le Context API de React, sans Redux. Trois contextes : `AuthContext` pour l'utilisateur connecté, `ThemeContext` pour l'apparence, `NotificationContext` pour les notifications. Redux aurait été surdimensionné : mon état global se limite à trois valeurs peu changeantes, l'essentiel de l'état est local aux pages. »

### ? « Comment gérez-vous les appels API ? »

« Une instance axios centrale avec deux intercepteurs. Celui d'entrée ajoute le jeton à chaque requête. Celui de sortie détecte le 401, purge la session et redirige vers la connexion. Résultat : **aucune page ne manipule le jeton ni ne gère l'expiration de session**. C'est centralisé en une quinzaine de lignes. Au-dessus, j'ai un service par domaine métier — 29 dans l'application Projets. »

### ? « Comment le TypeScript vous a-t-il aidé ? »

« Un seul fichier `types/index.ts` contient toutes les interfaces miroir des DTOs Java. Si le backend renomme un champ, je le change à un seul endroit et le compilateur me signale **toutes** les pages impactées au `npm run build` — le script fait `tsc -b` avant `vite build`, donc un build ne peut pas passer avec une erreur de type. »

### ? « Comment déployez-vous sur GitHub Pages, qui ne gère pas le routage client ? »

« Deux astuces. Le script de build copie `index.html` en `404.html` : quand l'utilisateur recharge `/projects/media-plan`, GitHub ne trouve pas de fichier et sert `404.html`, qui contient l'application entière et laisse React Router résoudre la route. Et la base de Vite est conditionnelle : `/` en local, `/Antigone_RH/projects/` sur GitHub Pages, avec `basename` passé à `BrowserRouter`. »

## ? « Le mode sombre est-il partagé entre les applications ? »

« Oui, via un **cookie** et non `localStorage`. En développement les trois applications tournent sur des ports différents : le `localStorage` est isolé par port, mais les cookies sont partagés par domaine. L'ordre de priorité au démarrage est : cookie partagé, puis `localStorage`, puis la préférence système `prefers-color-scheme`. »

## 6. Questions sur l'IA

(Réponses détaillées dans `05-IA-EXPLIQUEE-SIMPLEMENT.md §16` — voici les essentielles.)

### ? « Votre IA, c'est juste un appel à ChatGPT ? »

« Non. Un appel direct à GPT-4o ne connaîtrait rien de l'agence et inventerait les chiffres. Mon assistant a 12 outils Java qui interrogent la base PostgreSQL réelle, un moteur de recherche hybride sur les documents de l'entreprise, une mémoire conversationnelle avec résumé automatique, et surtout un contrôle d'accès qui s'applique **à chaque outil, avant chaque lecture**. »

### ? « Qu'est-ce que le “function calling” ? »

« Le modèle ne peut pas exécuter de code. Ce qu'il peut faire, c'est **dire** “je voudrais appeler `InvoiceLookupTool` avec `factureId=42`”. C'est mon code Java qui reçoit cette intention, vérifie les droits, exécute la requête et lui renvoie le résultat sous forme de texte. Le modèle n'a jamais la main sur la base. »

### ? « Comment empêchez-vous les hallucinations ? »

« Trois niveaux. Les prompts interdisent explicitement d'inventer un montant, une date ou une référence — toute donnée factuelle doit venir d'un outil. Les outils fournissent des montants déjà calculés que le modèle doit citer tels quels, jamais recalculer. Et pour la paie, je vais plus loin : après la génération, j'écrase les montants de la réponse par ceux de la base. Même si le modèle se trompait dans sa prose, les chiffres exposés par l'API sont ceux du bulletin. »

### ? « Un utilisateur peut-il manipuler l'IA pour voir des données interdites ? »

« Non, et pour une raison structurelle : l'identifiant de l'employé ne vient jamais du message, il vient du **JWT signé**. La méthode `resolvePay SlipEmployeeId` remplace systématiquement la valeur demandée par celle du jeton pour tout compte non administrateur, et refuse explicitement toute demande visant un tiers en journalisant la tentative. Le texte du message n'a aucune autorité — la seule source d'identité est le jeton signé. Et les prompts ne sont qu'une couche de confort ; la sécurité réelle est dans le code Java. »

### ? « Qu'est-ce que le RAG et pourquoi “hybride” ? »

« RAG signifie Retrieval-Augmented Generation : avant de répondre, on va chercher les documents pertinents et on les injecte dans le prompt. “Hybride” parce que j'utilise deux méthodes de recherche en parallèle. La recherche vectorielle comprend les synonymes — “lancement” retrouve “teasing nouvelle collection” — mais dilue les identifiants exacts. La recherche par mots-clés retrouve précisément un numéro de facture mais ne comprend aucun synonyme. Je fusionne les deux par Reciprocal Rank Fusion : je fusionne les **rangs**, pas les scores, parce que `ts_rank_cd` et la similarité cosinus vivent sur des échelles incomparables. »

### ? « Combien coûte l'IA en API ? »

« Trois optimisations. L'indexation est incrémentale : chaque contenu est haché en SHA-256, seuls les chunks modifiés sont réembeddés — une réindexation nocturne coûte un embedding par ligne changée. La mémoire est résumée au-delà de 20 messages au lieu d'être renvoyée intégralement. Et un quota de 20 requêtes par minute et par compte empêche qu'une boucle côté client consomme le budget. »

## ? « Et si l'API OpenAI tombe ? »

« L'application démarre et fonctionne normalement. Les beans LLM sont conditionnels : sans clé API, ils ne sont pas créés, et seuls les endpoints `/api/v1/**` répondent 503. Tout le backend RH, Projets et Finance est intact. L'IA est une fonctionnalité additionnelle, jamais une dépendance critique. »

## 7. Questions sur le métier

### ? « Expliquez le calcul de la paie tunisienne. »

« Du brut vers le net : on part du salaire brut, on applique les primes et les déductions d'absences pour obtenir le brut effectif. On retire la CNSS salarié pour obtenir le salaire imposable. On applique l'abattement pour obtenir le revenu net imposable. Sur cette base, on calcule la contribution de solidarité et l'IRPP selon un barème progressif à 8 tranches, appliqué sur le revenu annualisé puis divisé par 12. Le net est le brut effectif moins ces trois retenues, et le net à payer déduit encore les acomptes. En parallèle, côté employeur : CNSS patronale, TFP, FOPROLOS et accidents du travail. »

### ? « Pourquoi une dichotomie pour passer du net au brut ? »

« Parce qu'en Tunisie on négocie souvent un salaire net, alors que le calcul va du brut vers le net. L'IRPP étant progressif par tranches, la relation n'est pas inversible analytiquement : chaque tranche a son propre taux, et changer le brut peut faire changer de tranche. J'ai donc implémenté une recherche dichotomique sur 64 itérations, entre le net comme borne basse et le triple du net comme borne haute, avec une convergence à 0,0005 dinar. Chaque itération recalcule tout le net à partir du brut candidat. »

### ? « Le barème IRPP est-il codé en dur ? »

« Non, aucun taux n'est codé en dur. Le barème est une entité `BaremeIrpp` avec un champ `effectiveFrom` et une liste de tranches stockée en JSON, et les taux CNSS, solidarité, abattement, TFP, FOPROLOS et AT sont dans `ParametresPaie`. Les deux sont éditables depuis l'interface Finance. C'est indispensable : les taux tunisiens changent par loi de finances presque chaque année. Le champ `effectiveFrom` permet même de préparer le barème de l'année suivante sans casser le calcul des bulletins passés. »

### ? « Comment calculez-vous la TVA ? »

« La TVA n'est due que sur ce qui est **encaissé**, pas sur ce qui est facturé. Pour chaque facture du mois, je calcule la proportion réellement payée et je n'applique la TVA qu'à cette proportion. Une facture payée à 40 % ne génère que 40 % de sa TVA. J'ajoute la TVA des autres revenus, je soustrais la TVA déductible issue des charges, et j'obtiens la TVA nette à payer. »

### ? « Comment calculez-vous les jours de congé ? »

« Le backend parcourt chaque date entre le début et la fin. Il exclut les jours non travaillés selon l'horaire affecté à l'employé — l'horaire est une chaîne "LUNDI,MARDI,..." donc c'est configurable par employé — et les jours fériés de la table `calendrier`. Il applique aussi des règles particulières comme le cas des demandes ne couvrant que des vendredis. Le calcul est fait côté serveur et exposé par un endpoint dédié, appelé en direct pendant la saisie : l'utilisateur voit immédiatement combien de jours seront décomptés. »

## ? « Pourquoi ce calcul côté serveur et pas dans le navigateur ? »

« Parce qu'il dépend de données serveur — les jours fériés de l'entreprise, l'horaire affecté à cet employé, les types de congé paramétrés — et surtout parce que le résultat doit être **rigoureusement identique** à celui utilisé au moment de l'approbation. Deux implémentations différentes finiraient par diverger. »

## ? « Comment fonctionne l'agent de bureau ? »

« C'est une application Electron installée sur le poste de l'employé. Elle détecte l'activité via `powerMonitor.getSystemIdleTime`, avec un seuil d'inactivité à 2 minutes, et envoie un heartbeat au backend toutes les 60 secondes avec l'IP, le SSID Wi-Fi et l'état actif ou inactif. Le principe fondamental, écrit dans le code : **l'agent collecte des données brutes, tout le métier est côté backend**. L'agent ne décide de rien. C'est modifiable sans réinstaller sur 30 postes, et non contournable en modifiant le client. »

## ? « Que se passe-t-il si l'employé éteint son PC sans se déconnecter ? »

« Un scheduler tourne toutes les 5 minutes. Pour chaque pointage du jour sans heure de sortie, si le dernier heartbeat date de plus de 10 minutes, il renseigne l'heure de sortie — et il prend **l'heure du dernier heartbeat**, pas l'heure courante. C'est l'heure réelle de départ, pas celle où le système s'en est aperçu. »

## 8. Questions sur la méthode

### ? « Comment avez-vous organisé le développement ? »

« Par domaines fonctionnels successifs. J'ai commencé par le socle — authentification, employés, rôles et permissions — parce que tout le reste en dépend. Puis le module RH avec les congés. Puis Projets et Media Plan. Puis Finance. Puis l'agent de bureau. Et enfin l'assistant IA, qui consomme tous les modules précédents. Le module IA a d'ailleurs remplacé deux chatbots séparés existants — un pour le règlement intérieur sur Groq/Llama, un pour la paie — en les fusionnant sous un routage d'intention automatique. »

### ? « Comment testez-vous ? »

« Côté backend, la configuration Maven sépare les tests unitaires, exécutés par Surefire sur `mvn test` sans dépendance externe, et les tests d'intégration, exécutés par Failsafe sur `mvn verify` avec Testcontainers et un vrai PostgreSQL. J'ai choisi Testcontainers plutôt que H2 parce que j'utilise des fonctionnalités spécifiques à Postgres — recherche plein texte française, pgvector, requêtes natives — qu'H2 ne supporte pas : un test qui passe sur H2 ne prouverait rien. Côté frontend, le widget IA a Vitest, Testing Library, MSW pour simuler les réponses HTTP, et Playwright pour l'end-to-end. »

### ? « Comment avez-vous déployé ? »

« Le backend en Docker sur Render, région Frankfurt, avec une base PostgreSQL 16 managée. La configuration est déclarative dans `render.yaml`, avec toutes les variables sensibles en `sync: false`. Les trois frontends sont construits par GitHub Actions à chaque push sur `main` et publiés sur GitHub Pages, dans des sous-dossiers `/rh/`, `/projects/` et `/finance/`. »

### ? « Quelle est la partie dont vous êtes le plus fier ? »

*Choisissez celle que vous maîtrisez le mieux. Exemple :*

« L'isolation du contexte utilisateur dans les outils IA. Le piège évident aurait été un `ThreadLocal`, et ça aurait fonctionné en apparence — jusqu'au jour où un outil échoue avant son nettoyage, et où l'utilisateur suivant hérite du contexte du précédent sur un thread mutualisé. C'est une fuite de données entre comptes, silencieuse et très difficile à reproduire. Comprendre le modèle d'exécution de LangChain4j et choisir d'instancier les outils par requête m'a pris du temps, mais ça rend le cloisonnement structurel : il ne dépend d'aucune discipline de nettoyage. »

## 9. Questions pièges

⚠ « **Votre application ne fait-elle pas doublon avec un ERP du marché ?** »

« Un ERP généraliste ne gère pas le media plan, qui est le livrable central d’une agence de communication, ni les rectifs, ni le calendrier de tournage, ni le portail client sur Drive. Il ne gère pas non plus nativement la paie tunisienne avec son barème IRPP progressif, la contribution de solidarité, le FOPROLOS et la TFP. Le paramétrer coûterait plus cher que ce développement, et l’agence resterait dépendante d’un éditeur pour chaque évolution métier. »

⚠ « **Pourquoi ne pas avoir utilisé une architecture microservices ?** »

« Parce que ça aurait été une complexité non justifiée. Les microservices résolvent des problèmes de mise à l’échelle indépendante et d’équipes multiples. Ici, il y a une seule base de données, un développeur et une trentaine d’utilisateurs. Un monolithe modulaire — avec des packages bien séparés, notamment le module IA qui est totalement autonome — donne la même clarté sans la latence réseau, la complexité de déploiement et les transactions distribuées. Si le besoin apparaissait, le module IA serait le premier candidat à l’extraction : il ne dépend du métier qu’en lecture. »

⚠ « **Le monitoring des employés, ce n’est pas une atteinte à la vie privée ?** »

Question éthique probable. À préparer sérieusement.

« C’est une question légitime, et j’ai fait des choix explicites. L’agent ne capture ni écran, ni frappe clavier, ni contenu : il envoie uniquement un état actif/inactif, l’IP et le SSID — le strict nécessaire pour établir un pointage. Ensuite, aucun automatisme ne sanctionne : chaque rapport d’inactivité passe par une décision managériale, parce qu’une inactivité peut être une réunion, un appel ou un déplacement. Enfin, l’agent croise les congés, autorisations et télétravail approuvés pour ne jamais compter une absence justifiée. C’est un outil de pointage, pas un outil de surveillance comportementale. »

⚠ « **52 tables, ce n’est pas surdimensionné ?** »

« Elles couvrent quatre domaines métier complets : RH avec les congés, production avec les projets et media plans, finance avec la facturation et la paie, et monitoring. Chaque domaine à lui seul justifierait une application. Et plusieurs tables sont des tables de jointure ou des journaux d’historique, qui sont peu nombreuses en colonnes mais indispensables à la traçabilité. »

⚠ « **Que se passerait-il avec 500 utilisateurs au lieu de 30 ?** »

« Trois points bloqueraient. Le rate limiter de l’IA est en mémoire : en multi-instances il deviendrait un quota par instance, il faudrait passer sur Redis. La recherche dense en repli calcule les cosinus en Java : au-delà de quelques dizaines de milliers de chunks, pgvector deviendrait obligatoire. Et le plan gratuit de Render ne tiendrait pas la charge. En revanche l’architecture elle-même tient : l’authentification est stateless, donc horizontalement scalable, et la couche métier est sans état. »

## 10. Les limites à assumer

Les annoncer soi-même dans la conclusion est un signe de maturité.

| Limite                        | Ce que je dirais                                                                                                                                          |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ddl-auto: update</code> | « En production il faudrait Flyway ou Liquibase avec des migrations versionnées. Hibernate ne sait ni supprimer une colonne ni modifier une contrainte. » |
| Rate limiter en mémoire       | « Documenté dans le code : en multi-instances, le quota deviendrait par                                                                                   |

|                      |                                                                                                                                                                                 |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | instance. Redis serait la suite logique. »                                                                                                                                      |
| Couverture de tests  | « Les tests couvrent le widget IA et les points critiques du backend, mais la couverture globale mérite d’être étendue, en priorité sur PayrollCalculator et DemandeService . » |
| Secret dans le dépôt | « J’ai identifié une clé API en dur dans application.yml ; la remédiation est révocation, régénération, passage en variable d’environnement. »                                  |
| pgvector non garanti | « Sur une instance managée sans droits d’extension, je bascule sur un cosinus calculé en Java : fonctionnellement identique, moins performant à grande échelle. »               |
| Prompt injection     | « Un prompt n’est jamais une garantie absolue. C’est pourquoi la sécurité réelle est dans AiAccessScope , pas dans le prompt. »                                                 |
| Plan gratuit Render  | « Le service s’endort après inactivité, d’où un premier appel lent. C’est une contrainte d’hébergement, pas d’architecture. »                                                   |

Les perspectives d’évolution à proposer

- 1. **Migrations versionnées** (Flyway) — la priorité technique n°1
- 2. **Application mobile** pour les demandes de congé et la validation
- 3. **Export PDF** des bulletins de paie et des factures
- 4. **Tableaux de bord prédictifs** : prévision de charge, de trésorerie
- 5. **Extension du RAG** aux comptes rendus de réunion et aux briefs clients
- 6. **Signature électronique** des documents RH

11. Fiche mémo

Les chiffres à connaître par cœur

BACKEND  
Spring Boot 3.5.6 · Java 17 · PostgreSQL 16  
52 entités métier + 5 entités IA = 57 tables  
35 contrôleurs métier + 5 IA = 40  
44 services métier + ~10 IA  
98 DTOs · 20 énumérations · 26+ permissions  
7 tâches planifiées

FRONTENDS  
React 19 · TypeScript 5.9 · Vite 7 · Tailwind 4  
Projects : 46 pages · 13 dépendances · port 3000  
RH : 41 pages · 12 dépendances · port 3001  
Finance : 9 pages · 7 dépendances · port 3002

IA  
GPT-4o (chat) + text-embedding-3-small (1536 dimensions)  
LangChain4j 1.19.0  
4 capacités · 12 outils · 5 tables  
RAG : 12 + 12 candidats → 8 retenus · poids dense 0,6 · RRF k=60  
Mémoire : 20 messages · résumé tous les 6 · 150 mots max  
Quota : 20 requêtes / minute / compte  
Heartbeat SSE : 15 s

PAIE  
Barème IRPP : 8 tranches, de 0 % à 40 %  
Dichotomie NET → BRUT : 64 itérations, convergence 0,0005 DT  
Contrats exonérés : CIVP, Freelance, Stage  
Solde congé : 18 j la 1re année, 24 j ensuite  
Échéance CNSS : le 15 du mois suivant la fin du trimestre

RELANCES  
≤ 7 jours → SOFT  
≤ 30 jours → FIRM  
> 30 jours → FORMAL

Les 6 phrases à placer absolument

- 1. « Le frontend cache, le backend interdit. »
- 2. « L’agent collecte, le backend décide. »
- 3. « L’IA propose, l’humain valide. »
- 4. « Aucune décision d’accès n’est déléguée au prompt. »
- 5. « Aucun taux fiscal n’est codé en dur — la loi de finances change chaque année. »

## 12. Checklist avant la soutenance

---

### Technique — la veille

- ☐ Le backend démarre et se connecte à la base
- ☐ Les trois frontends démarrent ( `npm run dev:projects` , `dev:rh` , `dev:finance` )
- ☐ La clé OpenAI est valide (tester une génération complète)
- ☐ Des **données de démonstration réalistes** sont en base : au moins 2 clients avec fiche remplie, 3 employés, des media plans passés (indispensable pour que l'IA ait un historique à exploiter), une facture en retard de plus de 30 jours, des bulletins de paie générés
- ☐ Le compte de démonstration a les bonnes permissions
- ☐ Swagger UI répond : `http://localhost:8080/swagger-ui.html`
- ☐ **Plan B** : captures d'écran ou vidéo de chaque étape de la démo, au cas où la connexion internet ou l'API OpenAI ferait défaut

### Documents

- ☐ Rapport imprimé
- ☐ Présentation exportée en PDF (au cas où le format ne passerait pas)
- ☐ Ces fiches de préparation relues

### Répétition

- ☐ La démonstration faite **au moins deux fois en entier**, chronométrée
- ☐ Les 3 points techniques forts racontés **à voix haute**, sans notes
- ☐ Un ordinateur de secours ou une clé USB avec le projet

### Le jour J

- ☐ Arriver en avance, tester l'affichage et la connexion
- ☐ Backend **démarré avant** le début de la présentation (le premier appel est toujours le plus lent)
- ☐ Les trois onglets déjà ouverts, connecté
- ☐ Respirer. Vous connaissez ce projet mieux que quiconque dans la salle.